

Fine Tuning



Anurag Kumar
27/02/2026

The Query

Who won the Euro 2026 World Championship?

User Query: Recent Event

The Failure

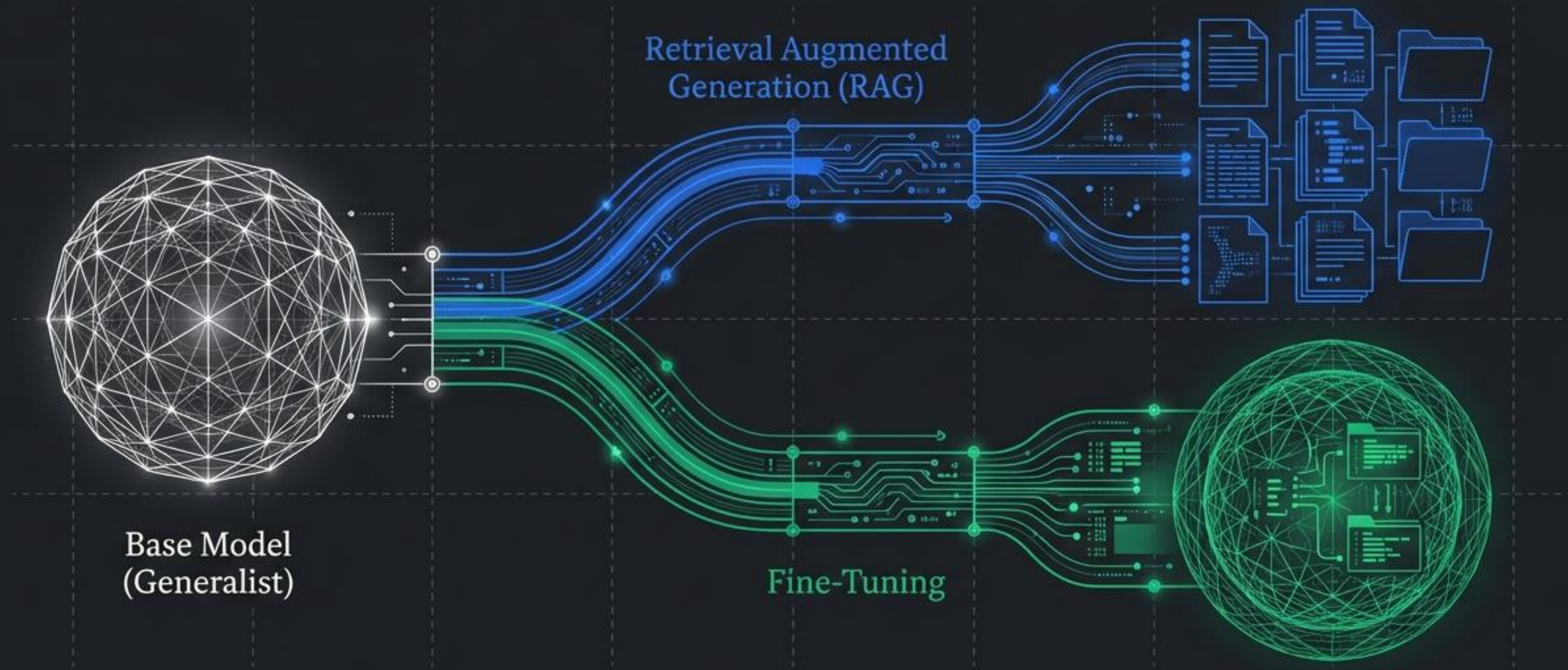
I cannot answer this question accurately based on my training data.



The Limitation: Base models are frozen at their training cutoff. They lack context, recent history, and access to private enterprise data.

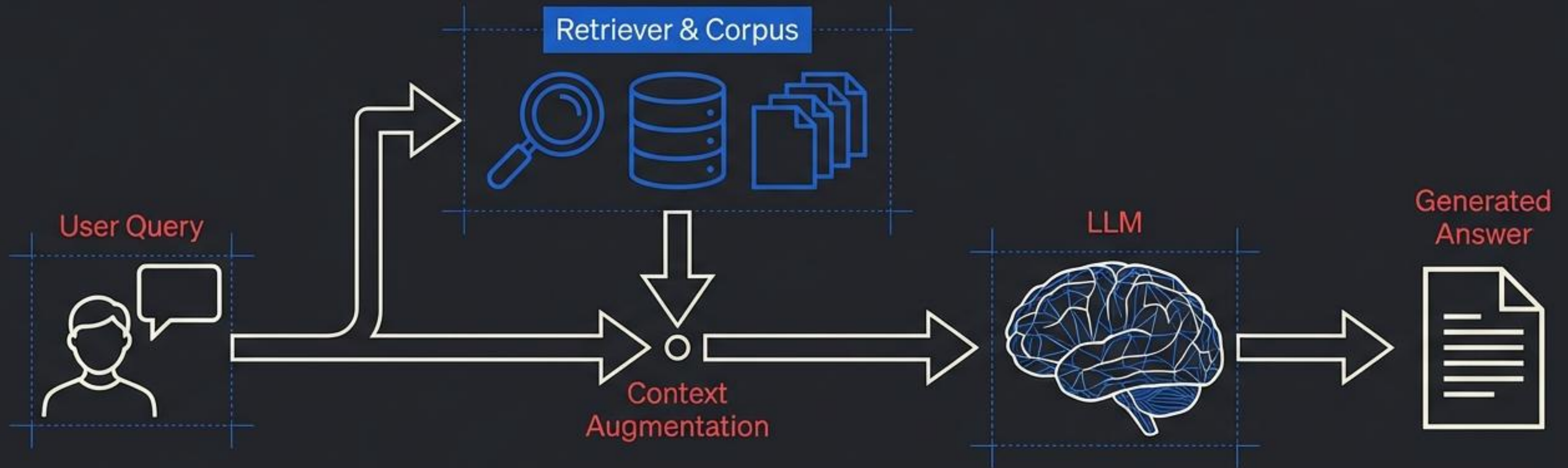
Beyond the Base Model

Strategies for adapting Large Language Models to Enterprise Data.



Solution A: Retrieval Augmented Generation (RAG)

The 'Open Book' Approach



Mechanism

Supplements the model externally.

Training

No weight updates required.

Data Source

PDFs, Databases, Live Feeds.

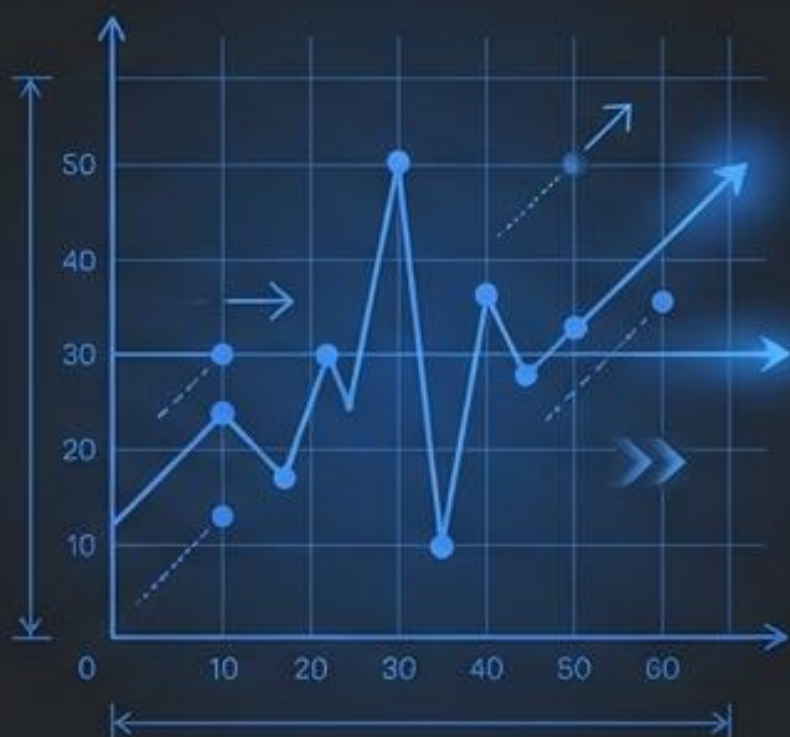
Solution B: Fine-Tuning



Key Mechanic: Fine-tuning bakes information internally into the model's weights. It alters behavior, style, and tone—creating a specialist (e.g., a *legal aide*) rather than just a generalist with a textbook.

Decision Point 1: Data Velocity

Fast & Dynamic



Best for data that updates continuously (News, Stock Prices, Inventory). RAG pulls fresh data in real-time without retraining.

Slow & Static



Best for fixed knowledge bases (Legal Precedents, Medical Codes, Historical Records). Knowledge is 'cut off' after training.

Decision Point 2: Trust & Transparency



Source: Document B, Page 4

RAG: High Transparency

Reduces hallucinations by grounding answers in retrieved proofs. Users can verify sources.



Fine-Tuning: Low Transparency

The answer is 'baked in' to the weights. It is difficult to trace exactly why the model generated a specific response.

Strategic Application: Real-World Use Cases



Product Doc Chatbot

Strategy: RAG.
Needs constant updates
on new features.



Legal Summarizer

Strategy: Fine-Tuning.
Needs to master
"Legalese" and specific
formatting.



Retail Assistant

Strategy: RAG.
Needs real-time access to
inventory levels.

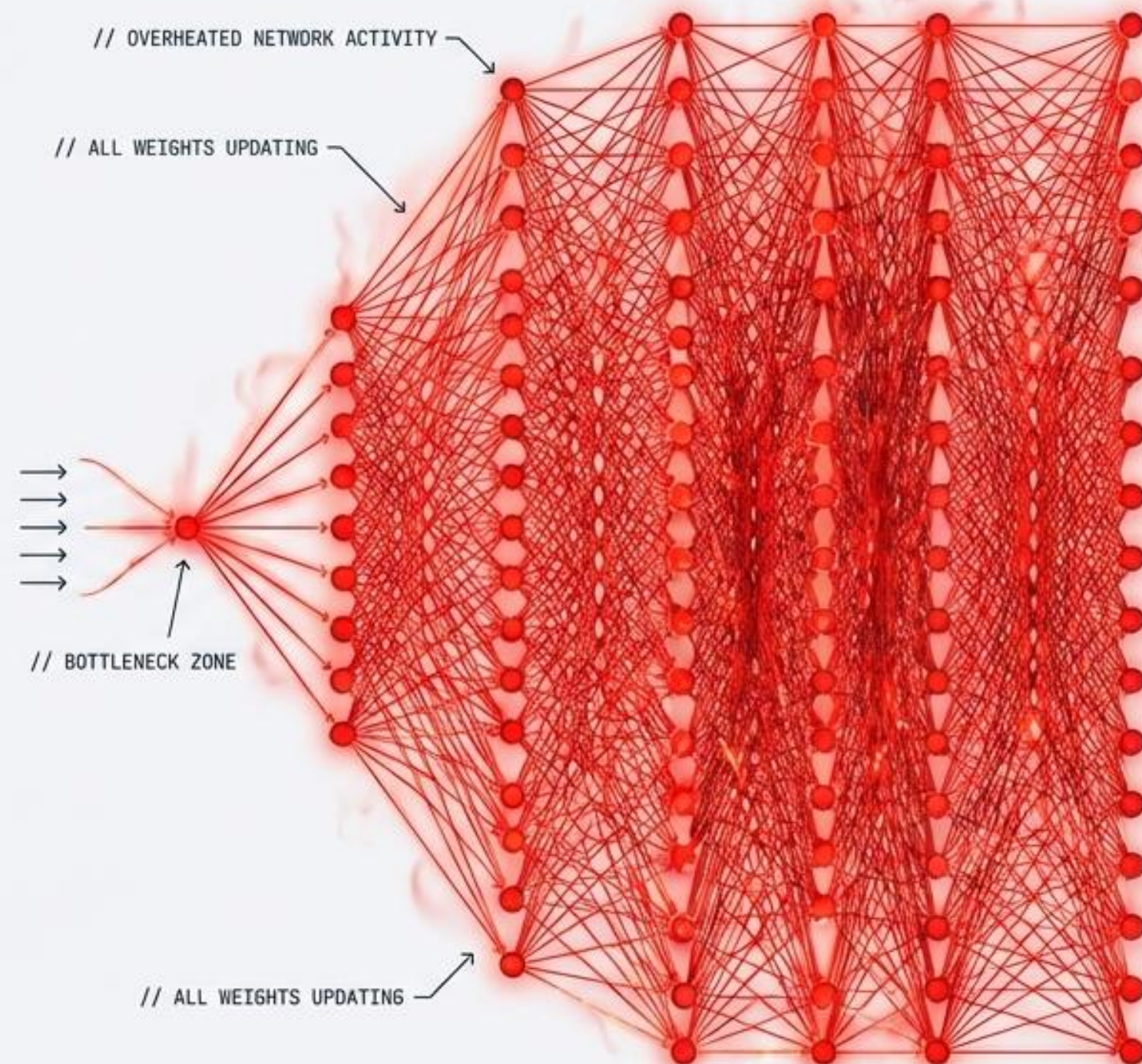
Way of fine tuning



The Bottleneck: Full Parameter Fine-Tuning

// Model: 175 Billion Parameters
// Requirement: Update 175 Billion Weights
// Result: Massive GPU VRAM Consumption

Traditional fine-tuning requires updating every single weight in the model. This creates a massive hardware barrier, making deployment and iteration prohibitively expensive.



PARAMETER-EFFICIENT FINE-TUNING(PEFT)

Definition: Freezes most of the **pre-trained weights** and tunes only a **small subset of Parameters**.

Benefits:

- Drastically reduces compute requirements
- Lower memory and storage costs
- Faster training and deployment
- Makes fine-tuning accessible on limited hardware



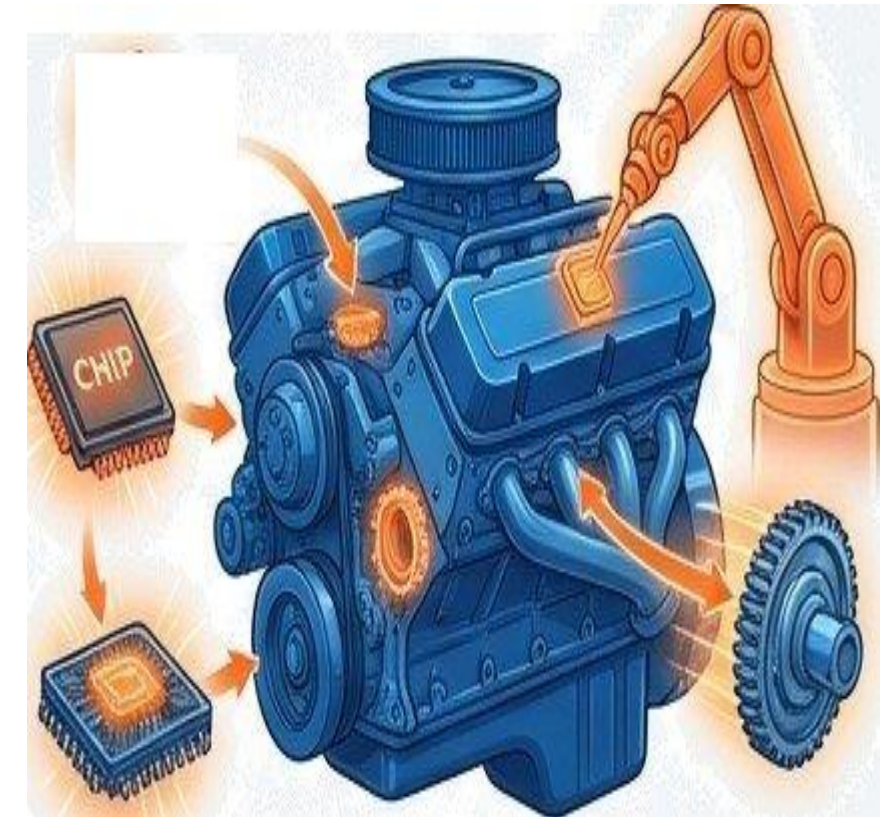
Domain Specific

Adapting for a sector.
Example: Finance, Sales,
Retail.



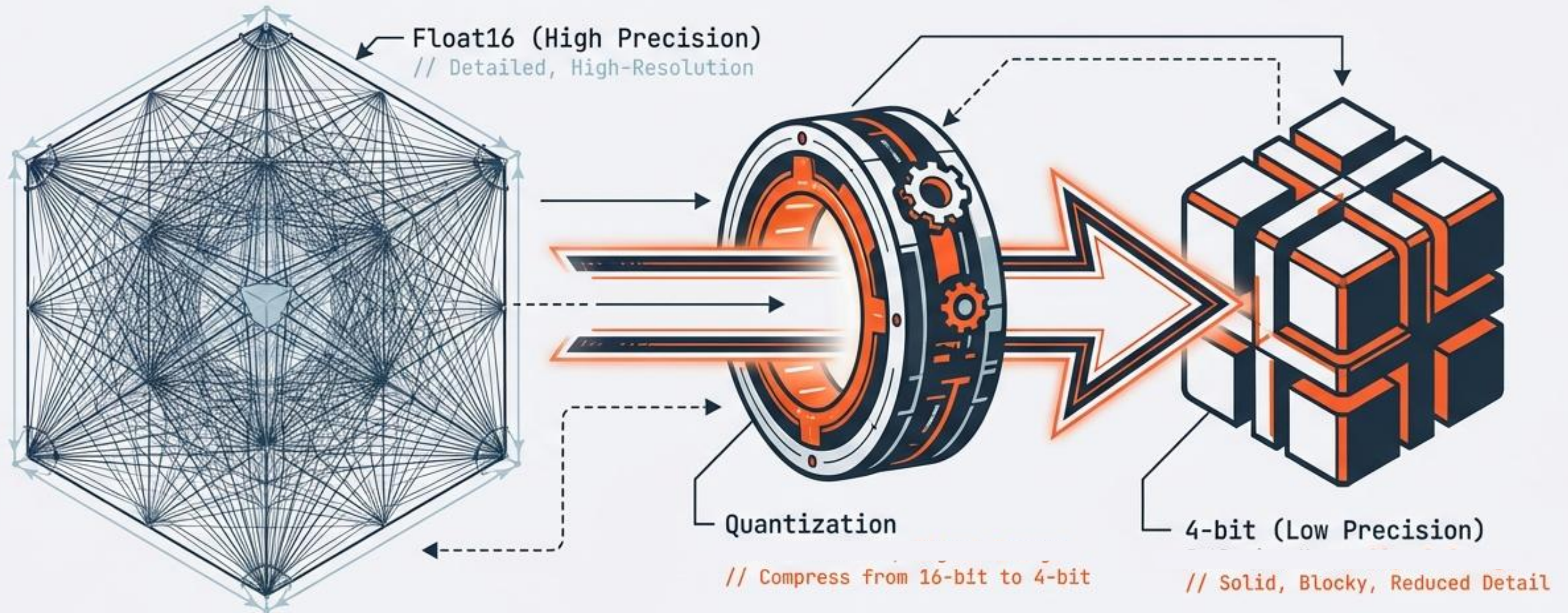
Specific Task

Teaching behavior.
Example: Q&A, Text-to-SQL.



QLoRA: Efficiency through Quantization

LoRA reduces trainable parameters. QLoRA reduces the memory footprint of the base model itself by converting weights from 16-bit to 4-bit precision.



LoRA

LoRA (Low-Rank Adaptation) freezes the pre-trained model weights and learns small low-rank matrices that modify existing weight matrices.

How it Works

1. The massive base model weights (W_0) remain unchanged.
2. During fine-tuning, only small low-rank matrices A and B are trained.
3. For inference, the outputs of the original weights and the low-rank update BA are combined.

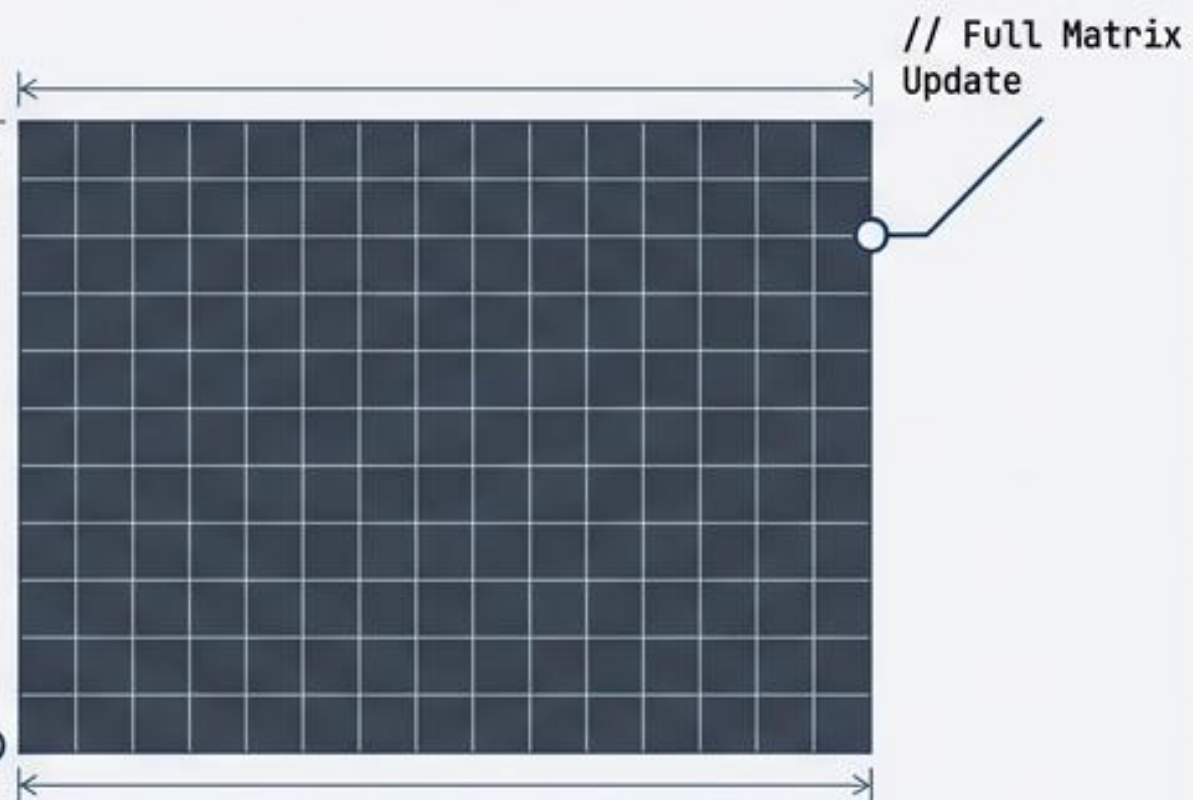
$$W = W + B * A$$

Frozen Pre-trained
Weights


Trainable LoRA
Adapter Weights



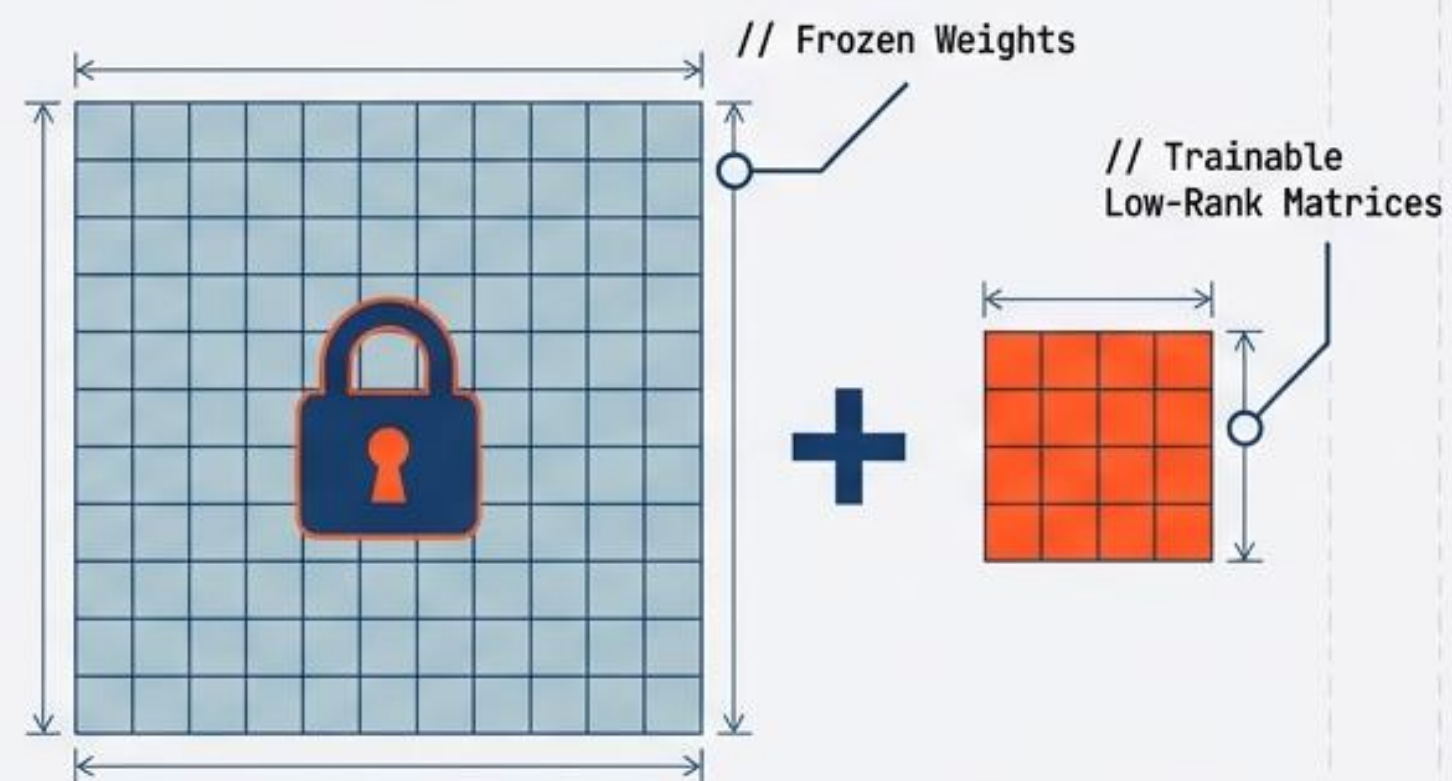
Traditional



Update W (All Weights)

// JetBrains Mono

LoRA Approach



W0 (Frozen)

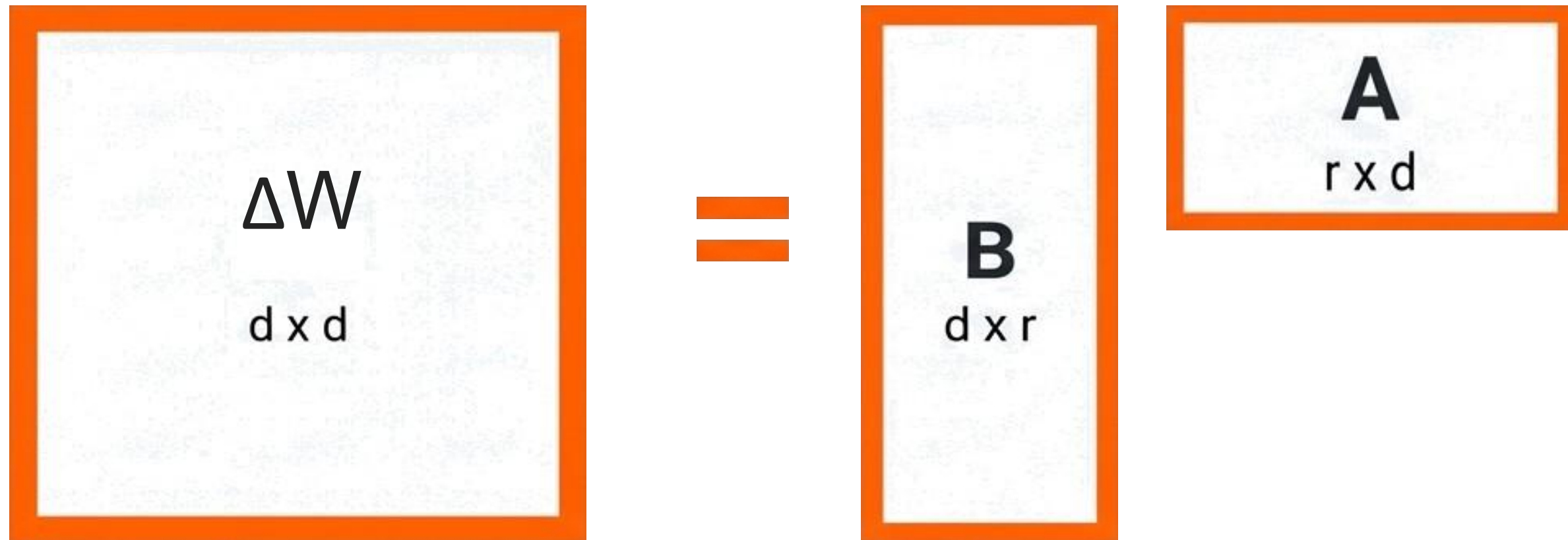
Delta W
(Trainable Changes)

LoRA freezes the massive pre-trained weights and only trains the changes in a separate, tiny set of matrices.

$$W_{\text{new}} = W_{\text{frozen}} + \text{Delta}_W$$

The Math Behind the Magic: Low-Rank Decomposition

The Power of LoRA: Instead of training a **weight update** matrix, LoRA decomposes the update into two smaller matrices.



The diagram illustrates the Low-Rank Decomposition (LoRA) concept. It shows a large square matrix ΔW with dimensions $d \times d$ on the left. This matrix is equal to the product of two smaller matrices, B and A , on the right. Matrix B is a tall, narrow rectangle with dimensions $d \times r$. Matrix A is a short, wide rectangle with dimensions $r \times d$. All three matrices are enclosed in orange borders, and an equals sign is placed between them.

$$\Delta W_{d \times d} = B_{d \times r} A_{r \times d}$$

Parameters to train: $2 * d * r$ instead of $d * d$

The Power of Rank (r)

Rank is a hyperparameter you control. Even at Rank 64, trainable parameters are $< 1\%$ of the original model size.



// Parameter Count Comparison

SCALE: NON-LINEAR

Define the LoRA Configuration

Objective: Use the **PEFT library** to define how **LoRA adapters** are integrated and applied to the model.

```
from peft import LoraConfig
```

```
lora_config = LoraConfig(  
    r=8,  
    lora_alpha=16,  
    target_modules=["q_proj", "v_proj"],  
    lora_dropout=0.1,  
    bias="none",  
    task_type="CAUSAL_LM",  
)
```

The **rank of the update matrices**—a key **hyperparameter** that determines the **size and capacity** of the A and B.

A **scaling factor** for the LoRA weights, serving a role **similar to a learning rate**.

Apply **LoRA adapters** specifically to the **query, and value projection layers** within the attention mechanism.

Specify the model's **task type** to ensure **correct configuration**.

All Together with the SFTTrainer

Use the **SFTTrainer** from **TRL** to manage the **supervised fine-tuning loop** with the **LoRA configuration**.

```
from trl import SFTTrainer
trainer = SFTTrainer(
    model=model,
    Train_dataset=dataset,
    peft_config=lora_config,
    Max_seq_length=512,
    tokenizer=tokenizer,
    args=training_args,
)
trainer.train()
```

Our **LoRA** configuration.

Standard training arguments like batch size, epochs, learning rate, etc.

Start the fine-tuning process.

The Efficiency Hierarchy

QLoRA offers the optimal balance, enabling high-performance fine-tuning on consumer-grade hardware.

Method	Weights Updated	Memory Usage	Hardware Needed
Full Fine-Tuning	100%	Massive	Data Center GPU (A100)
LoRA	< 1%	Standard	High-End Workstation
QLoRA	< 1%	Lowest (4-bit)	Consumer GPU (RTX 3090/4090)

Thank You.

Anurag Kumar